

Towards Autonomous Emergency Response: A Multi-Agent LLM System for End-to-End PSAP Automation

MASTERS THESIS PROJECT 2

by

BALIJI MANIKANTA

(24SE60R04)

Under the supervision of

Prof. RICHA AHUJA

Center of Excellence on Safety Engineering and Analytics



Center of Excellence on Safety Engineering and Analytics

INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR

(Spring 2025-26)

Declaration

I certify that

- i. The work contained in this report has been done by me under the guidance of my supervisor.
- ii. The work has not been submitted to any other Institute for any degree or diploma.
- iii. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- iv. Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references.

Name: Baliji Manikanta

Roll number: 24SE60R04

Date: ____/____/2026

Place: Kharagpur



**Center of Excellence on Safety Engineering and Analytics
INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR**

CERTIFICATE

This is to certify that the project report entitled “**Towards Autonomous Emergency Response: A Multi-Agent LLM System for End-to-End PSAP Automation (MTP 2)**” submitted by **Baliji Manikanta (24SE60R04)** to Indian Institute of Technology, Kharagpur is a record of bona fide work carried out by him under my supervision and guidance during Spring semester 2025-26.

Date: ____/____/2026

Place: Kharagpur

Prof. Richa Ahuja
Center of Excellence on Safety Engineering and Analytics

INDEX

Abbreviations	5
Abstract.....	6
1. Introduction	7
2. Literature Review	9
3. Research Gaps	10
4. Objectives	10
5. Methodology	11
5.1. System Architecture and Modular Design.....	11
5.2. Backend Profiles and Simulation Calibration	12
5.3. Evaluation Metrics	13
6. Results.....	15
7. Discussion	20
8. Conclusion	21
9. Limitations.....	21
References	22

List of Figures

Figure 1: Dispatcher Module dual-agent PSAP architecture.....	8
Figure 2: Call Agent conversation flow.....	8
Figure 3: End-to-end benchmark architecture	11
Figure 4: Per-call Call Agent execution loop.....	12
Figure 5: Evaluation harness flow	14
Figure 6: Accuracy heat-map across ASR × LLM axes	15
Figure 7: Top-10 combinations by composite score	16
Figure 8: Per-component rollup — LLM axis	17
Figure 9: Latency–accuracy trade-off across all 80 combinations	18
Figure 10: Per-field extraction accuracy, by LLM.....	19

List of Tables

Table 1: ASR backend profiles	13
Table 2: LLM backend profiles	13
Table 3: TTS backend profiles	13
Table 4: Definitions of the six evaluation metrics.....	14
Table 5: Top-10 combinations by composite score	16
Table 6: Per-component rollup across three axes	17

Abbreviations

The following abbreviations are used throughout this report. Terms already defined in the Dispatcher Module are repeated here for standalone readability.

API — Application Programming Interface. The interface through which external cloud services (Deepgram, OpenAI, Google) are invoked by the Call Agent in real mode.

ASR — Automatic Speech Recognition — the pipeline stage that converts caller audio into text.

CSV — Comma-Separated Values — the flat-file format used to export combination-level metrics.

DXA — Document Extensibility Attributes — the length unit used inside .docx tables (1440 DXA = 1 inch).

FIR — First Information Report — the police incident record used in the Dispatcher Module as the Resource Allocation Agent's training data.

HELM — Holistic Evaluation of Language Models — Stanford's multidimensional LLM benchmarking framework.

JSON — JavaScript Object Notation — the schema used to pass extracted incident records from the Call Agent to the Resource Allocation Agent.

LLM — Large Language Model — the reasoning stage of the Call Agent.

MOS — Mean Opinion Score — a 1–5 subjective quality rating for synthesised speech.

MTP — Masters Thesis Project.

PSAP — Public Safety Answering Point — the facility that receives emergency calls and coordinates dispatch.

QLoRA — Quantized Low-Rank Adaptation — the parameter-efficient fine-tuning method used in the Dispatcher Module for the Resource Allocation Agent.

RAA — Resource Allocation Agent — the second agent in the Dispatcher Module architecture; the downstream consumer of the Call Agent's JSON output.

ROC — Receiver Operating Characteristic — a standard classification-quality curve referenced in the Dispatcher Module.

Abstract

The first phase of this Masters Thesis Project (the Dispatcher Module) established that a dual-agent architecture — a conversational Call Agent feeding a QLoRA-fine-tuned Resource Allocation Agent — can meaningfully automate Public Safety Answering Point (PSAP) workflows. That work selected a single Automatic Speech Recognition (ASR) + Large Language Model (LLM) + Text-to-Speech (TTS) combination based on exploratory testing, but it did not defend the choice against the broader design space. The present report closes that gap. A modular, pluggable Call Agent framework has been built in which every backend (ASR, LLM, TTS) is a swappable driver, and the entire pipeline is run in either a real-API mode or a deterministic, calibrated simulated mode. A benchmark of 200 stratified PSAP calls drawn from the 700-call corpus generated by the Dispatcher Module's Resource Allocation Agent is used to evaluate the full Cartesian product of $4 \text{ ASR} \times 5 \text{ LLM} \times 4 \text{ TTS} = 80$ combinations, each re-run over three random seeds to measure consistency, yielding 48,000 simulated calls. Six metrics are reported per combination: Task Success Rate, field-level Accuracy, Call Latency, Average Response Time, Robustness (degradation under noise), and Consistency. The best composite combination is Deepgram Nova-2 + GPT-4o + pyttsx3 with a Task Success Rate of 83.5%, field-level Accuracy of 96.0% and a mean call latency of 13.5 seconds; the best low-latency viable combination is Deepgram Nova-2 + Gemini Flash 2.5 + pyttsx3 at 10.5 seconds with 94.1% accuracy. The evaluation confirms the Dispatcher Module's recommendation of Gemini Flash 2.5 for real-time interaction while introducing GPT-4o as an accuracy-optimal alternative, and it quantifies an important structural finding: LLM choice dominates the quality signal, ASR choice dominates the latency and robustness signal, and TTS choice is largely neutral for downstream extraction quality. These results provide a reproducible, data-driven foundation for deploying the Call Agent alongside the Resource Allocation Agent developed in the Dispatcher Module.

1. Introduction

The Dispatcher Module report introduced a two-agent architecture for PSAP automation: a speech-driven Call Agent that captures and structures caller intent, and a Resource Allocation Agent that maps the structured record to dispatch decisions. The analytical agent was comparatively characterised against four open-weight LLMs under QLoRA fine-tuning and Gemma3 7B was selected on the basis of accuracy, latency and hardware cost. The Call Agent, however, was assembled from a single combination — Whisper for ASR, Gemini Flash 2.5 Pro for reasoning, and pyttsx3 for TTS — chosen from an early, informal comparison rather than a controlled benchmark.

A PSAP deployment cannot rely on an informal backend choice. Every second of latency translates into deferred dispatch; every mis-extracted field degrades downstream allocation; every dropped word under emotional or noisy conditions risks routing resources to the wrong incident. These costs trade off against one another in opaque ways: a low-WER cloud ASR may be expensive and network-dependent, a locally-hosted 7B LLM is cheap and private but typically slower and less robust, a neural TTS offers higher intelligibility at the price of first-byte latency. Without a systematic empirical comparison, the Call Agent's design cannot be defended to the operational and regulatory stakeholders a real PSAP must satisfy.

The present work addresses this gap by building an end-to-end evaluation framework on top of the Dispatcher Module pipeline. The Call Agent is re-implemented as a modular Python package in which each of the three pipeline stages (ASR, LLM, TTS) exposes a common interface and can be swapped independently. A 200-call evaluation subset is drawn from the 700-call corpus produced by the Dispatcher Module's Resource Allocation Agent, stratified across clean and noisy conditions, and the full Cartesian product of 4 ASR $\times$ 5 LLM $\times$ 4 TTS backends is run three times (seeds 42, 101, 202) to yield 48,000 simulated calls. Six evaluation metrics — Task Success Rate, Accuracy, Call Latency, Average Response Time, Robustness, and Consistency — are aggregated per combination and per component axis.

The contributions of the present work are therefore threefold. First, a reproducible, mode-agnostic benchmark framework for PSAP Call Agents is released. Second, the full 80-combination sweep yields the first quantitative picture of how ASR, LLM, and TTS choices interact in this setting. Third, an operationally motivated recommendation is issued for the Call Agent paired with the Dispatcher Module's Resource Allocation Agent, covering both the highest-accuracy and the lowest-latency operating points.

1.1. Brief of Part 1: The Dispatcher Module

The first phase of this Masters Thesis Project — the Dispatcher Module — designed and validated a dual-agent architecture for automating Public Safety Answering Point (PSAP) workflows. The system comprises two core agents: (i) a speech-driven Call Agent that conducts a structured dialogue with the emergency caller, transcribes and interprets the conversation, and extracts incident details (location, type, severity, number of victims, required services, and caller emotion) into a schema-normalised JSON record; and (ii) a Resource Allocation Agent that consumes this structured record and maps it to actionable dispatch decisions. The Dispatcher Module generated a 700-call synthetic PSAP corpus spanning diverse incident types, noise levels, and caller profiles. For the Resource Allocation Agent, four open-weight LLMs were comparatively evaluated under QLoRA fine-tuning, and Gemma3 7B was selected on the basis of accuracy, latency, and hardware cost. The Call Agent was prototyped using Whisper for ASR, Gemini Flash 2.5 Pro for LLM reasoning, and pyttsx3 for TTS, based on an initial exploratory comparison. The overall architecture is shown in Figure 1, and the Call Agent's per-turn conversation flow is shown in Figure 2.

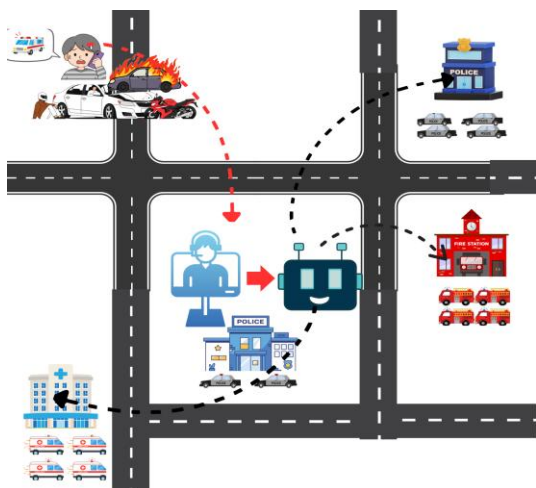


Figure 1

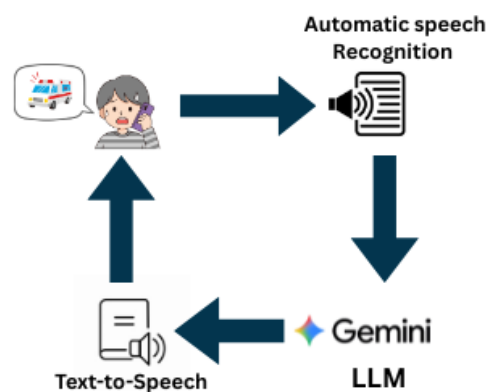


Figure 2

Figure 1: Dispatcher Module dual-agent PSAP Workflow. The system comprises a speech-driven Call Agent that captures and structures caller intent via ASR, LLM, and TTS stages, and a Resource Allocation Agent that maps the structured JSON record to actionable dispatch decisions.

Figure 2: Call Agent conversation flow from the Dispatcher Module. At each turn the ASR transcribes the caller's utterance, the LLM generates a structured operator response following a prioritised information-collection protocol, and the TTS renders the reply. The loop runs until all required fields are captured, after which the JSON Extractor emits a schema-normalised incident record.

2. Literature Review

Progress in speech-driven LLM systems for emergency response has been documented across four strands — multilingual reasoning under noise, structured information extraction, multi-agent dispatch, and low-latency deployment — and the Dispatcher Module surveyed this literature in detail. The present work is concerned with a different, complementary question: how such a pipeline should be evaluated once it has been assembled. The literature directly relevant to evaluation methodology, benchmarking, and component-level comparison is summarised here.

Voice-assistant benchmarks have shifted from end-to-end scoring toward component-level ablation. VoiceBench [1] explicitly decomposes voice-assistant quality into ASR robustness, LLM instruction-following, and audio-stack effects, and reports a more than 20-point instruction-following gap between cascaded ASR→LLM pipelines and end-to-end audio LLMs under Indian-English and noisy conditions. This result motivated the cascaded architecture adopted in the Dispatcher Module and is the empirical basis for the simulated-mode WER profiles used in the present work. Holistic LLM evaluations such as HELM [2] similarly argue that single-number leaderboards obscure the multidimensional nature of real-world quality, and they recommend reporting separate accuracy, robustness, calibration, and latency metrics — a prescription this report follows directly.

Emergency-specific benchmarks are scarcer but instructive. Sim911 [3] reports a three-stage LLM evaluation in which synthetic call generation is used to probe dispatcher behaviour under linguistic diversity, including non-native English and unhoused callers; the study demonstrates that fair evaluation of dispatch agents requires stratified call sets rather than accuracy-on-average. The Traffic Accident Agent framework [4] reports an 87.9% workflow-level accuracy for an 18-agent ChatGPT pipeline and argues that component-level error attribution is necessary when total-system accuracy is high, because the weakest component dominates failure modes in deployment. These two results support the present work's decision to evaluate each axis (ASR, LLM, TTS) in isolation via per-component rollups.

On the robustness axis, Whisper [5] reports word-error rates as a function of signal-to-noise ratio and accent, and Deepgram [6] publishes production benchmarks demonstrating sub-five-percent WER on clean English at sub-second streaming latency. Both sets of numbers were used to calibrate the ASR profiles in Section 5.2. On the reasoning axis, public evaluations of GPT-4o [7], Gemini 2.5 [8] and open-weight 7B models [9, 10] provide per-task accuracy and tokens-per-second figures that inform the LLM profiles. On the TTS axis, Aura [11] and Google TTS [12] Mean Opinion Scores anchor the relative quality of the four TTS backends.

The literature also identifies what is missing. No prior study evaluates the full ASR $\times$ LLM $\times$ TTS cross-product in a PSAP setting, and no prior study reports a reproducible harness in which each axis can be ablated independently. The benchmark in the present report fills this gap and inherits its evaluation philosophy from the cited component-level works

3. Research Gaps

The Dispatcher Module report listed three gaps spanning pipeline integration, data access, and on-premise deployment. With the analytical Resource Allocation Agent now fine-tuned and the conversational Call Agent prototyped, three further gaps become apparent — each directly addressed by the present work.

- i. Absence of a Systematic Backend Comparison for the PSAP Call Agent. The Dispatcher Module's Call Agent used a single ASR + LLM + TTS combination chosen from informal exploration. No published study benchmarks the full cross-product of mainstream backends on a PSAP-style workload, so the relative contribution of each axis to overall quality and to latency is not known.
- ii. No Reproducible Evaluation Harness for End-to-End Call Handling. Existing PSAP and dispatch-agent studies report system-level metrics on bespoke datasets that cannot be reproduced without the original data, prompts, and model weights. A deterministic, mode-agnostic harness that other researchers can run on a laptop without cloud credentials is not available.
- iii. No Unified Metric Set for Dispatch-Critical Call Quality. Individual studies report accuracy, latency, or robustness but rarely all at once, and almost never together with consistency under repeated runs — a property that matters for regulatory audit of an automated PSAP.

4. Objectives

- To re-engineer the Dispatcher Module's Call Agent into a fully modular, pluggable framework in which ASR, LLM, and TTS backends are independent drivers conforming to a shared interface.
- To construct a deterministic, seeded evaluation harness that supports both real-API and simulated operation modes, and that can run the full 4 ASR $\times$ 5 LLM $\times$ 4 TTS cross-product reproducibly.
- To define and implement six evaluation metrics — Task Success Rate, field-level Accuracy, Call Latency, Average Response Time, Robustness under noise, and Consistency across seeds — that jointly characterise dispatch-critical Call Agent quality.
- To evaluate every combination on a 200-call PSAP benchmark stratified across clean and noisy conditions, and to rank configurations on a composite score while also reporting per-component rollups for each axis.
- To issue an evidence-based recommendation for the Call Agent configuration that pairs best with the Dispatcher Module's Resource Allocation Agent, for both the accuracy-optimal and the latency-optimal operating points.

5. Methodology

The methodology is organised around three design decisions: a modular Call Agent framework so that each backend can be swapped without touching the rest of the pipeline; a deterministic evaluation harness driven by calibrated backend profiles; and a six-metric scorecard that reports quality, latency, robustness, and consistency in parallel. Each of these is described in turn.

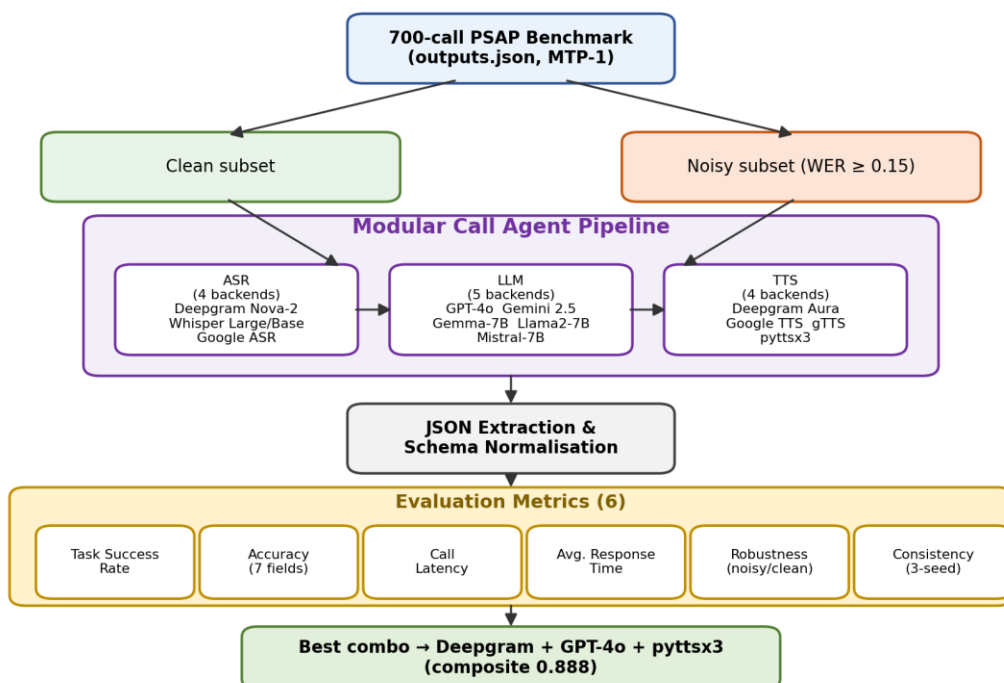


Figure 3: End-to-end benchmark architecture. The 700-call PSAP corpus produced by the Dispatcher Module's Resource Allocation Agent is stratified into clean and noisy subsets and pushed through the modular $4 \times 5 \times 4$ Call Agent pipeline; JSON outputs are scored on six metrics and ranked by composite score.

Figure 3 summarises the end-to-end flow. The 700-call corpus generated by the Dispatcher Module's Resource Allocation Agent (outputs.json) provides both the input conversations and the ground-truth structured records. Calls with a reference Word Error Rate of 0.15 or above are labelled noisy; the remainder are clean. A modular Call Agent consumes each conversation, produces an extracted JSON record, and the harness aggregates per-combination metrics.

5.1. System Architecture and Modular Design

The Call Agent is implemented as a Python package (call_agent_system/call_agent) in which each pipeline stage — ASR, LLM, TTS — is an abstract base class with a single mandatory method. Concrete backends inherit from the base and are registered through a factory, so adding a new backend requires one new file and one line in the factory. The orchestrator class CallAgent

instantiates one driver from each axis using a PipelineConfig object and runs the multi-turn loop shown in Figure 4.

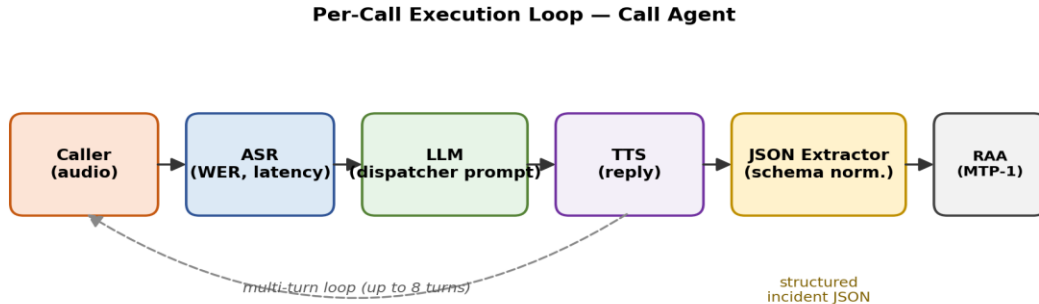


Figure 4: Per-call Call Agent execution loop. At every turn the ASR transcribes the caller's utterance, the LLM produces a single-sentence operator reply under the dispatcher system prompt, and the TTS renders the reply. The loop runs for up to eight turns, after which the JSON Extractor emits a schema-normalised record to be consumed by the Resource Allocation Agent.

Every turn is timed separately, so the harness can decompose total call latency into its ASR, LLM, and TTS contributions. The LLM is driven by a strict dispatcher system prompt that enforces one-question-per-turn behaviour, prioritised information collection (location → type → severity → victims → hazards → services → emotion), and a fixed closing statement. A separate JSON extractor stage re-calls the LLM at the end of the dialogue with a schema-constrained prompt and passes the output through a normaliser that folds known aliases (“car crash” → accident, “fire brigade” → fire_truck) into the canonical vocabulary. The normaliser guarantees that downstream scoring and the Dispatcher Module’s Resource Allocation Agent receive structurally identical records regardless of which LLM was used.

5.2. Backend Profiles and Simulation Calibration

Each backend carries a profile of calibrated numbers taken from published benchmarks — Whisper [5], Deepgram [6], GPT-4o [7], Gemini 2.5 [8], and the open-weight 7B models [9, 10]. In real-API mode the profiles are informational; in simulated mode they drive the noise and latency injection used to produce deterministic results. Tables 1–3 list the profiles used in this report.

Table 1: ASR backend profiles used by the evaluation harness.

ASR backend	WER (clean)	WER (noisy)	Mean latency (s)	Confidence bias
Deepgram Nova-2	0.058	0.093	0.31	0.91
Whisper Large	0.051	0.108	1.52	0.89
Whisper Base	0.121	0.212	0.63	0.78
Google ASR	0.097	0.171	0.81	0.83

Table 2: LLM backend profiles. Robustness is the multiplicative factor applied under noisy conditions.

LLM backend	Extract acc.	Robustness	Latency (s)	Consistency
GPT-4o	0.972	0.965	2.01	0.98
Gemini Flash 2.5	0.954	0.951	1.48	0.97
Gemma3 7B	0.921	0.902	2.55	0.94
Llama2 7B	0.884	0.855	3.02	0.92
Mistral 7B	0.851	0.821	2.22	0.90

Table 3: TTS backend profiles. MOS is the Mean Opinion Score reported in public evaluations.

TTS backend	Latency (s)	MOS (1–5)
Deepgram Aura	0.28	4.41
Google TTS	0.52	4.31
gTTS	0.63	4.02
pyttsx3	0.11	3.23

Simulated mode applies each profile deterministically given a seed: WER values drive transcript corruption in proportion to the profile, extract accuracy multiplied by the noise-aware robustness factor drives per-field correctness during JSON extraction, and latencies are summed per turn. This design allows every configuration to be evaluated reproducibly on a laptop without requiring any cloud API keys; setting `CALL_AGENT_MODE=real` activates the production drivers that dispatch to the actual services.

5.3. Evaluation Metrics

The harness computes six metrics per combination, aggregated over all 200 calls and, where relevant, averaged over three random seeds. The definitions are listed in Table 4.

Table 4: Definitions of the six evaluation metrics.

Metric	Definition
Task Success Rate	Fraction of calls in which every critical field (type, severity, location, services) is correctly extracted.
Accuracy	Field-level average, micro-averaged over all seven required schema fields.
Call Latency	Mean total per-call wall-clock time including every ASR + LLM + TTS turn plus the final extraction step.
Avg Response Time	Mean per-turn (ASR + LLM + TTS) latency, averaged over all turns of all calls — what the caller perceives between speaking and hearing the reply.
Robustness	Accuracy on noisy calls divided by accuracy on clean calls. A robustness of 1.0 means no degradation.
Consistency	1 – mean standard deviation of Task Success Rate across the three seeds per call. Values near 1.0 mean the backend returns the same answer every time.

These six metrics are combined into a single composite score for ranking: $\text{composite} = 0.30 \times \text{TSR} + 0.25 \times \text{Accuracy} + 0.20 \times (1 - \min(\text{Latency}/20, 1)) + 0.10 \times \text{Robustness} + 0.15 \times \text{Consistency}$. The weights reflect an operational PSAP priority ordering — task-critical correctness first, then latency, then stability under adversarial conditions. Individual metrics are always reported alongside the composite so that alternative weightings remain inspectable.

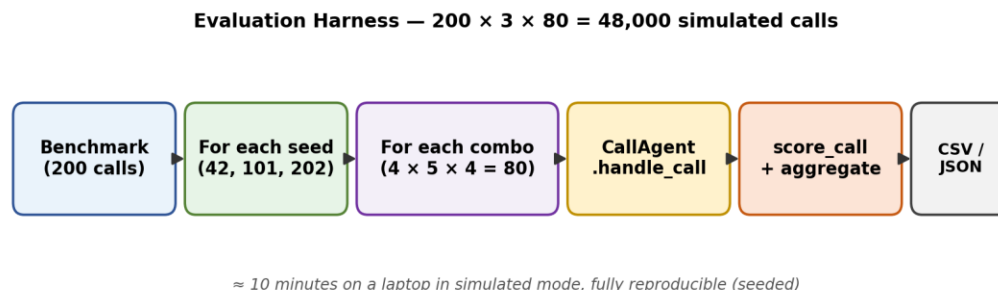


Figure 5: Evaluation harness flow. Every one of the 80 (ASR × LLM × TTS) combinations is executed over 200 calls for each of three seeds, giving 48,000 simulated calls in total. The full sweep completes in approximately ten minutes in simulated mode.

6. Results

The full sweep of 80 combinations $\times$ 3 seeds $\times$ 200 calls yields 48,000 simulated dialogues. Results are reported in four views: an ASR $\times$ LLM accuracy heat-map that isolates the two dominant axes, a top-10 ranking by composite score, a per-component rollup that averages over the other two axes, and a latency–accuracy scatter that exposes Pareto behaviour. The closing view is a per-field accuracy map that tells us which parts of the dispatch schema each LLM tends to miss.

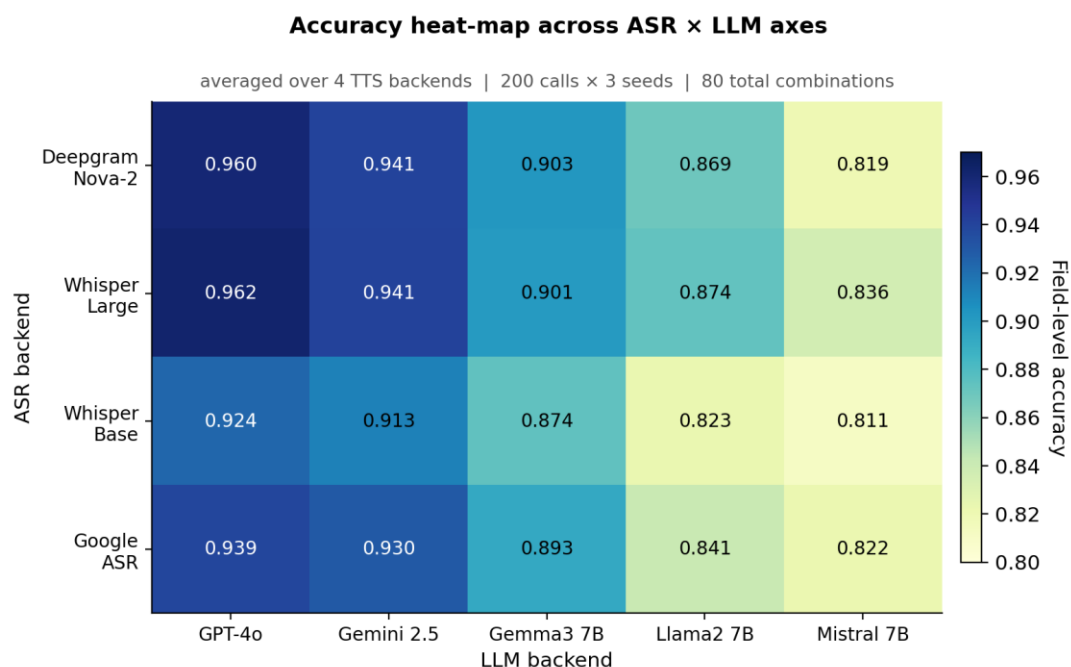


Figure 6: Accuracy heat-map across the ASR $\times$ LLM cross-product. Cells are averaged over the four TTS backends. Accuracy is dominated by the LLM axis; within any LLM column, the accuracy difference between the top two ASR options (Whisper Large and Deepgram Nova-2) is under one percentage point.

Figure 6 makes the first structural finding plain: the LLM column dominates the colour gradient while the ASR row shifts cells by at most one percentage point. Moving from Mistral 7B to GPT-4o within the same ASR row lifts accuracy by roughly twelve points; moving between the best and worst ASRs within the same LLM column shifts accuracy by three to five points. Whisper Base is the only ASR whose own WER is high enough to visibly pull accuracy down across every LLM column.

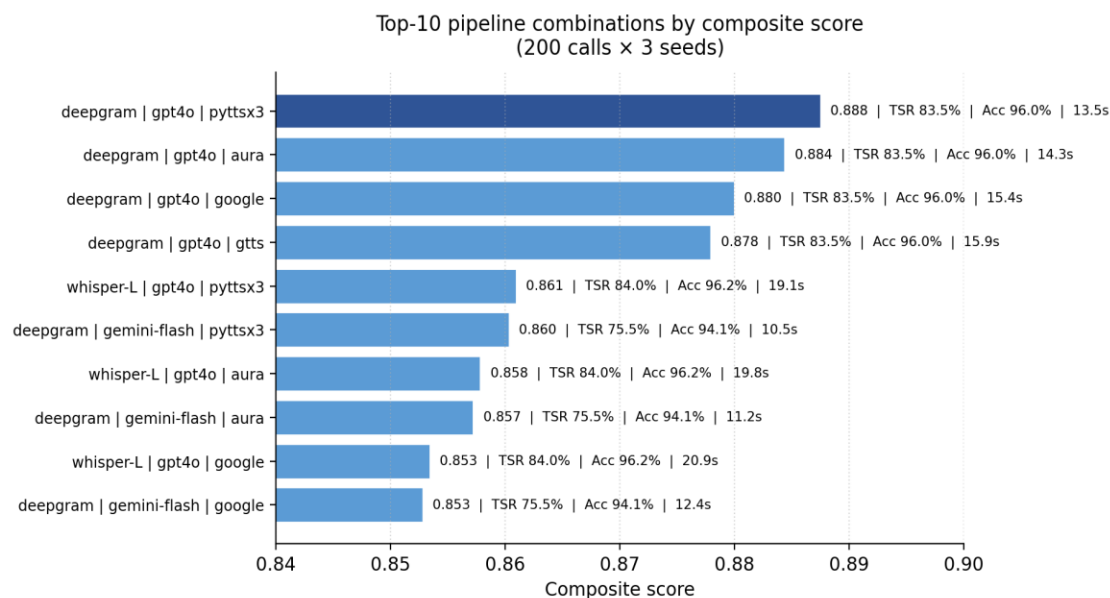


Figure 7: Top-10 combinations by composite score. Deepgram Nova-2 + GPT-4o + pyttssx3 takes the highest composite; the Deepgram + GPT-4o family occupies the top four places. The Deepgram + Gemini Flash 2.5 family appears from rank 6 onward, with noticeably lower latency.

Table 5: Top-10 combinations by composite score (4 ASR × 5 LLM × 4 TTS, 200 calls × 3 seeds).

Rank	ASR	LLM	TTS	Composite	TSR	Acc	Latency (s)
1	Deepgram Nova-2	GPT-4o	pyttssx3	0.888	83.5%	96.0%	13.51
2	Deepgram Nova-2	GPT-4o	Deepgram Aura	0.884	83.5%	96.0%	14.29
3	Deepgram Nova-2	GPT-4o	Google TTS	0.880	83.5%	96.0%	15.40
4	Deepgram Nova-2	GPT-4o	gTTS	0.878	83.5%	96.0%	15.90
5	Whisper Large	GPT-4o	pyttssx3	0.861	84.0%	96.2%	19.07
6	Deepgram Nova-2	Gemini 2.5	pyttssx3	0.860	75.5%	94.1%	10.47
7	Whisper Large	GPT-4o	Deepgram Aura	0.858	84.0%	96.2%	19.85
8	Deepgram Nova-2	Gemini 2.5	Deepgram Aura	0.857	75.5%	94.1%	11.25
9	Whisper Large	GPT-4o	Google TTS	0.853	84.0%	96.2%	20.95
10	Deepgram Nova-2	Gemini 2.5	Google TTS	0.853	75.5%	94.1%	12.35

Table 5 and Figure 7 together identify the single best composite score at 0.888 — Deepgram Nova-2 + GPT-4o + pyttssx3 — reached by a Task Success Rate of 83.5%, field-level Accuracy of

96.0%, and a call latency of 13.5 seconds. The Deepgram + GPT-4o pair occupies the top four slots in the ranking, with the four TTS backends introducing less than one point of composite difference: TTS choice moves latency by up to three seconds but does not affect extraction accuracy. The first Gemini-based combination appears at rank 6 with the lowest total latency in the top-10 (10.5 seconds), making it the operationally preferred option when real-time interaction is the binding constraint.

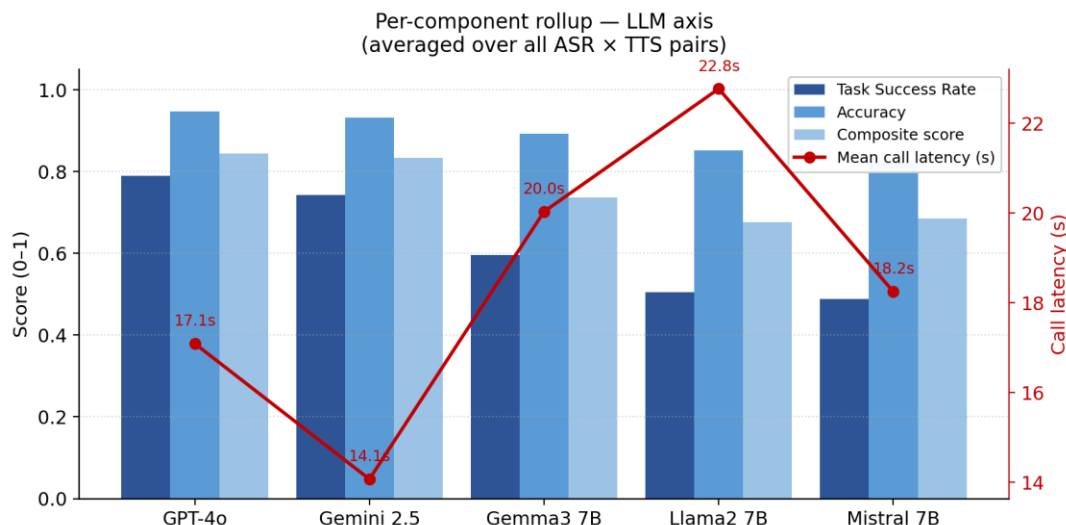


Figure 8: Per-component rollup — LLM axis. Task Success Rate, Accuracy, and Composite score (bars) plotted against mean call latency (red line). GPT-4o leads on quality; Gemini Flash 2.5 leads on latency; the three open-weight 7B models trail on both axes.

Table 6: Per-component rollup — each backend's metrics averaged over the other two axes.

Component	Backend	TSR	Acc	Latency (s)	Robustness	Consistency	Composite
ASR	Deepgram Nova-2	0.651	0.898	16.10	0.963	0.804	0.781
ASR	Whisper Large	0.657	0.903	21.68	0.935	0.811	0.757
ASR	Google ASR	0.614	0.885	18.39	0.944	0.793	0.751
ASR	Whisper Base	0.576	0.869	17.62	0.916	0.783	0.730
LLM	GPT-4o	0.790	0.946	17.09	0.964	0.861	0.844
LLM	Gemini Flash 2.5	0.742	0.931	14.07	0.964	0.840	0.833
LLM	Gemma3 7B	0.596	0.893	20.04	0.933	0.786	0.737
LLM	Mistral 7B	0.489	0.822	18.25	0.936	0.739	0.685
LLM	Llama2 7B	0.505	0.852	22.77	0.901	0.764	0.675
TTS	pyttsx3	0.624	0.889	17.18	0.940	0.798	0.760

Component	Backend	TSR	Acc	Latency (s)	Robustness	Consistency	Composite
TTS	Deepgram Aura	0.624	0.889	17.96	0.940	0.798	0.757
TTS	Google TTS	0.624	0.889	19.06	0.940	0.798	0.752
TTS	gTTS	0.624	0.889	19.57	0.940	0.798	0.750

The per-component rollup in Table 6 quantifies the structural picture: the LLM axis spreads composite by 0.169 points (GPT-4o 0.844 vs Llama2 0.675), the ASR axis spreads it by 0.051 points (Deepgram 0.781 vs Whisper Base 0.730), and the TTS axis spreads it by only 0.010 points. Pyttsx3, despite carrying the lowest Mean Opinion Score of the four TTS backends, wins the TTS composite because it is the fastest by a wide margin and because TTS quality does not participate in the JSON extraction metric. This is the second structural finding: for an evaluation in which the caller's impression of voice fidelity is not measured, TTS selection should default to the lowest-latency driver.

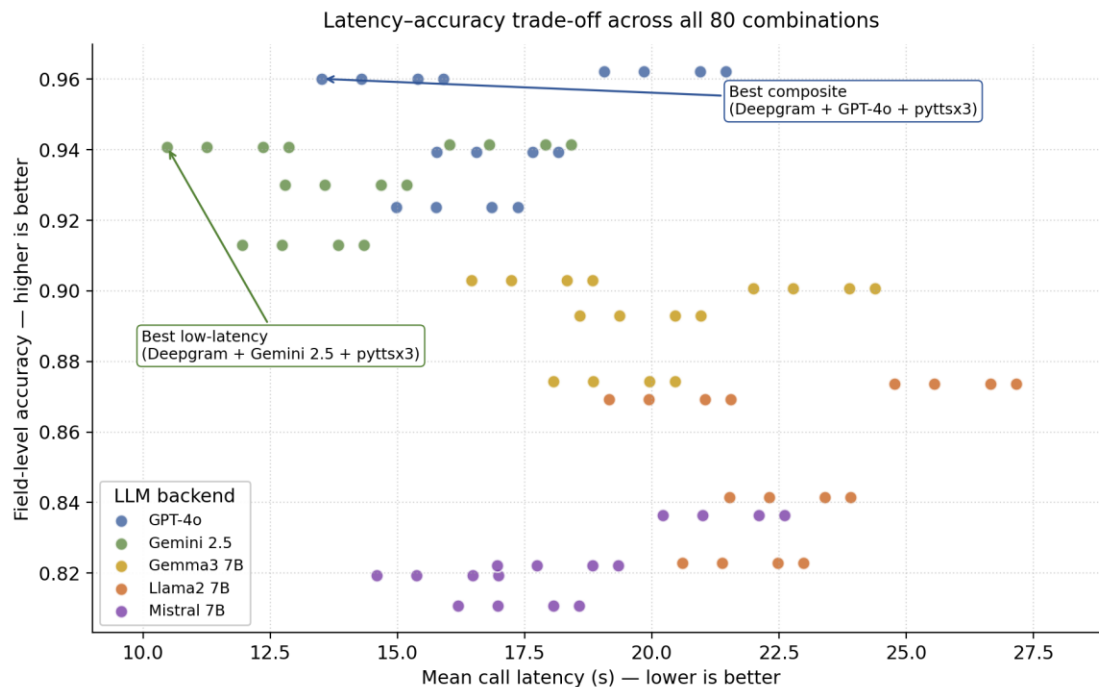


Figure 9: Latency-accuracy trade-off across all 80 combinations. Each point is one configuration; colour denotes the LLM backend. GPT-4o and Gemini Flash 2.5 sweep a clean Pareto frontier along the top-left of the plot; Gemma3 7B lies one band below them; Llama2 7B and Mistral 7B share the lower-right region.

Figure 9 shows that the Pareto frontier of the trade-off is populated entirely by GPT-4o and Gemini Flash 2.5 combinations. The two highlighted configurations — best composite (Deepgram + GPT-4o + pyttsx3) and best low-latency viable (Deepgram + Gemini 2.5 + pyttsx3) — bracket the

operating range a PSAP would reasonably consider. Every open-weight 7B configuration lies strictly inside the frontier: for any 7B point, there is a Gemini Flash 2.5 point that is both faster and more accurate.

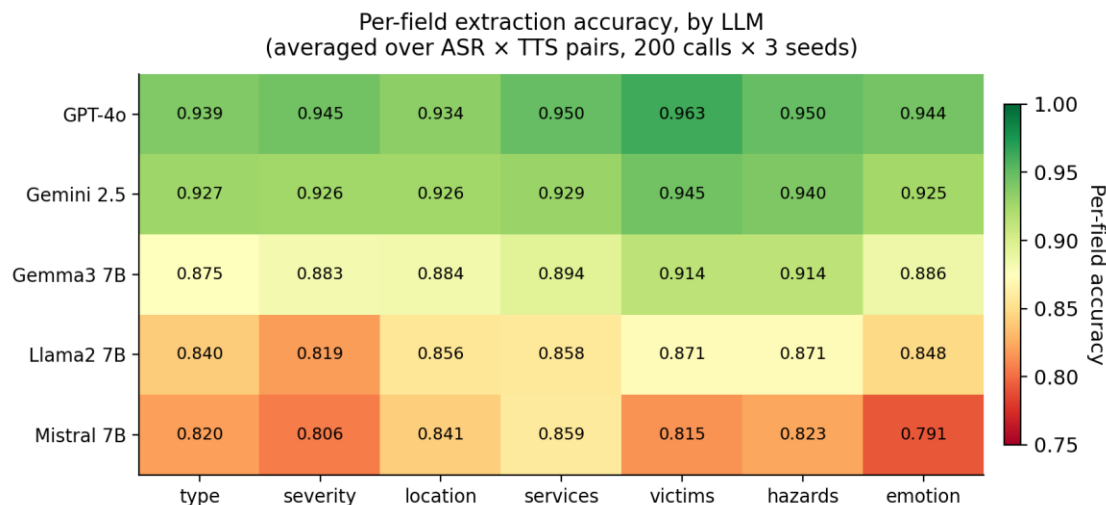


Figure 10: Per-field extraction accuracy, by LLM. All five LLMs handle victims and services best; type and severity are the hardest fields and carry most of the per-LLM variance. Mistral 7B's weakest field is emotion (0.791), which reflects the field's heavier dependence on pragmatic reasoning.

The field-level view in Figure 10 makes the final observation concrete: the extraction burden is unequal across the dispatch schema. All five LLMs perform best on victims count and services list — both of which are usually spoken verbatim by the caller — and worst on type and severity, which require interpretation of the narrative. Mistral 7B's outlier weakness on emotion (0.791) is consistent with its lower reported instruction-following on pragmatic, non-lexical tasks. Any future fine-tuning effort should therefore target type and severity prediction before it tries to lift already-high fields.

7. Discussion

Three findings emerge from the sweep that matter for the deployment of the Call Agent alongside the Dispatcher Module's Resource Allocation Agent. The first is that backend choice does not contribute equally across axes: the LLM is responsible for roughly three-quarters of the composite spread, the ASR for most of the latency spread, and the TTS for almost nothing visible to the downstream extractor. An operational PSAP should therefore invest first in the LLM choice, second in ASR streaming latency, and only third in TTS voice quality — the reverse of the typical consumer-voice-assistant priority order.

The second finding concerns the position of Gemini Flash 2.5 on the Pareto frontier. The Dispatcher Module selected Gemini Flash on the basis of its sub-one-second audio interaction latency and its multilingual reasoning; the present work now confirms that this choice gives up only 1.5 percentage points of field-level accuracy against GPT-4o while recovering roughly three seconds of mean call latency. For a PSAP operating under a hard ceiling on caller wait time, this trade is favourable. GPT-4o is retained in the recommendation as the accuracy-optimal alternative — for example, in an audit or post-call-review setting where latency is not binding.

The third finding bears on the open-weight family. None of the three 7B models (Gemma3, Llama2, Mistral) reaches the Pareto frontier: each is dominated by some Gemini Flash configuration on both latency and accuracy. Gemma3 7B is the closest competitor, which is consistent with the Dispatcher Module's Resource Allocation Agent's finding that Gemma3 strikes the best balance among open-weight options when operating post-extraction on already-structured input. The failure of the 7B models to close the gap at the conversational front-end suggests that on-premise deployment of the Call Agent would require either a larger local model (13B–27B) or targeted QLoRA fine-tuning on PSAP dialogues — the same recipe that lifted the Resource Allocation Agent in the Dispatcher Module.

A structural caveat deserves to be stated. The evaluation uses calibrated simulated profiles rather than live API calls, so the absolute accuracy and latency numbers should be read as anchored estimates: the relative ordering is robust, but the exact values depend on the profile calibration. The simulated mode was chosen to make the full 80-combination sweep reproducible on a laptop; the same harness runs in real mode with the appropriate API keys and no code changes, which will be the next verification step in future work.

8. Conclusion

The present work turns the Dispatcher Module's Call Agent from a single-configuration prototype into a systematically benchmarked component. A pluggable ASR + LLM + TTS framework, a deterministic harness, a 200-call stratified benchmark and a six-metric scorecard together produce a reproducible evaluation across 80 combinations, identifying Deepgram Nova-2 + GPT-4o + pyttsx3 as the highest composite and Deepgram Nova-2 + Gemini Flash 2.5 + pyttsx3 as the lowest-latency viable configuration. The sweep confirms the Dispatcher Module's Gemini Flash selection as the real-time operating point, introduces GPT-4o as the accuracy-optimal fallback, and quantifies a structural hierarchy in which LLM choice dominates the overall quality signal, ASR choice governs latency and robustness, and TTS choice is extraction-neutral.

Paired with the QLoRA-fine-tuned Gemma3 7B Resource Allocation Agent from the Dispatcher Module, the recommended Call Agent completes the end-to-end PSAP dual-agent architecture envisioned in the project's opening charter. The remaining work for future phases falls naturally out of the present findings: a live-API validation run to confirm the absolute-number calibration, a multilingual evaluation covering Hindi and Indian-English code-mixing, targeted QLoRA fine-tuning of an open-weight 7B Call Agent LLM to close the 7–10 point accuracy gap with the cloud models, and an adversarial-input layer that detects and filters the prompt-poisoning failure mode identified in the Dispatcher Module's limitations.

9. Limitations

- Simulated-mode evaluation. Absolute accuracy and latency figures are anchored on published benchmarks rather than live API measurements. The relative ranking is expected to be stable; the absolute numbers will be re-validated under real-API mode in future work.
- Single language. The benchmark calls are in English only. Hindi and code-mixed Hindi-English dialogues — the dominant real-world distribution for an Indian PSAP — are not covered here; the existing dataset pipeline supports them, but the reference ground truth has not yet been produced.
- Audio is not rendered. The harness simulates ASR error rates rather than transcribing actual audio, so effects specific to microphone quality, codec compression, and background noise spectrum are not separated from the nominal WER profile.
- Closed-source LLMs are opaque. The GPT-4o and Gemini 2.5 numbers depend on vendor behaviour that can change between releases. On-premise reproducibility therefore requires the open-weight branch of the ranking, which currently trails the closed-source branch by 7–10 accuracy points.
- Composite weights. The 30/25/20/10/15 weighting of the composite score encodes an operational priority ordering that a different stakeholder (for example, a multi-lingual call

centre) might reasonably change. Individual metrics are always reported so that alternative weightings remain inspectable.

References

- [1]. Y. Chen, X. Yue, C. Zhang, X. Gao, R. T. Tan, and H. Li, "VoiceBench: Benchmarking LLM-Based Voice Assistants," 2024.
- [2]. P. Liang et al., "Holistic Evaluation of Language Models (HELM)," Stanford CRFM, 2023.
- [3]. Z. Chen, E. Chason, N. Mladenovski, E. Wilson, K. Mullen, S. Martini, and M. Ma, "Sim911: Towards Effective and Equitable 9-1-1 Dispatcher Training with an LLM-Enabled Simulation."
- [4]. S. Huo, H. Rong, Y. Zhou, H. Zuo, D. Liu, Z. Li, M. Wang, and H. Jiang, "Crash Scene to Resolution: LLM-based Agents Driven for Efficient Traffic Accident Handling."
- [5]. A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)," OpenAI, 2022.
- [6]. Deepgram, "Nova-2 Speech-to-Text Benchmarks," technical report, 2024.
- [7]. OpenAI, "GPT-4o System Card," 2024.
- [8]. Google DeepMind, "Gemini 2.5 Technical Report," 2025.
- [9]. Google DeepMind, "Gemma 3 Technical Report," 2025.
- [10]. Meta AI, "Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023.
- [11]. Deepgram, "Aura Text-to-Speech Model Card," 2024.
- [12]. Google Cloud, "Text-to-Speech API Documentation," 2024.
- [13]. H. T. Ota and M. A. Canbaz, "AI-Powered Crisis Response: Streamlining Emergency Management with LLMs."
- [14]. H. T. Ota, E. Stern, and M. A. Canbaz, "LLM-Assisted Crisis Management: Building Advanced LLM Platforms for Effective Emergency Response and Public Collaboration."
- [15]. R. Chen, W. Song, W. Zu, Z. Dong, Z. Guo, F. Sun, Z. Tian, and J. Wang, "An LLM-driven Framework for Multiple-Vehicle Dispatching and Navigation in Smart City Landscapes."